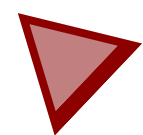


SMART CAR FOLLOW SIGNS PROJECT



ศุภเมธ จันทรสกุล 63050176

บุญชู เจิ้ง 63050618

สาขาระบบสมองกลฝังตัว

คำนำ

ฉบับนี้ เทคโนโลยียานยนต์ขับเคลื่อนอัตโนมัติ (Autonomous Vehicles) ได้รับการพัฒนาอย่างก้าวกระโดด และถูกมองว่าเป็นปัจจัยสำคัญในการยกระดับระบบขนส่งและโลจิสติกส์ในอนาคต อย่างไรก็ตาม การพัฒนารถยนต์ไร้คนขับขนาดจริงนั้นมีค่าใช้จ่ายและความเสี่ยงด้านความปลอดภัยสูง การสร้างระบบจำลองเพื่อทดสอบและพัฒนาอัลกอริทึมจึงเป็นแนวทางที่จำเป็น

นักศึกษาและวิศวกรศึกษาและวิเคราะห์ปัญหาทางวิศวกรรม คณะผู้จัดทำพบว่าระบบจำลองที่มีอยู่ยังคงมีข้อจำกัด โดยเฉพาะอย่างยิ่งในส่วนของความสามารถในการจดจำและตอบสนองต่อป้ายจราจร ได้อย่างรวดเร็วและแม่นยำ ซึ่งเป็นหัวใจสำคัญของการขับเคลื่อนอัตโนมัติที่ปลอดภัย

ฉบับนี้ คณะผู้จัดทำจึงได้จัดทำ แบบรายงานทางเทคนิค โครงการ "รถเดินตามป้ายจราจร" ฉบับนี้ขึ้น โดยมีวัตถุประสงค์เพื่อพัฒนาและสร้างรถขนาดเล็กที่ใช้ระบบสมองกลฝังตัว (Embedded System) Raspberry Pi 5 และเทคนิคการประมวลผลภาพขั้นสูง YOLOv8 (Deep Learning) ในการตรวจจับป้ายจราจรและสั่งการให้รถปฏิบัติตามได้อย่างถูกต้องตามข้อกำหนดที่ตั้งไว้

ฉบับนี้ครอบคลุมรายละเอียดตั้งแต่การวิเคราะห์และกำหนดที่มาของปัญหา ทฤษฎีพื้นฐานที่เกี่ยวข้อง (การประมวลผลภาพและการควบคุม) การออกแบบและการพัฒนา ไปจนถึงผลการทดสอบโครงการ หวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์และเป็นแนวทางในการศึกษาเพื่อพัฒนาเทคโนโลยีด้านยานยนต์ขับเคลื่อนอัตโนมัติให้มีประสิทธิภาพมากยิ่งขึ้นต่อไป

ส่วนที่ 1



- 1 ปัญหาและความสำคัญ
- 2 วัตถุประสงค์และขอบเขต

ปัญหาและความสำคัญ

ลักษณะและที่มาของปัญหาทางวิศวกรรม

การพัฒนาระบบขนส่งในยุคปัจจุบันมีความจำเป็นต้องพึ่งพา เทคโนโลยียานยนต์ขับเคลื่อนอัตโนมัติมากขึ้น หากการพัฒนาเหล่านี้หยุดชะงักจะส่งผลกระทบให้เกิดความล่าช้าและนำไปสู่ความสูญเสียทางเศรษฐกิจจากการขนส่งที่ขาดประสิทธิภาพได้

ปัญหาหลักทางวิศวกรรม

- **ค่าใช้จ่ายและความเสี่ยงสูง** : การพัฒนารถยนต์ไร้คนขับขนาดจริงมีค่าใช้จ่ายสูงมาก และมีความเสี่ยงด้านความปลอดภัยอย่างยิ่ง
- **ข้อจำกัดของระบบจำลองที่มีอยู่** : จากการสืบค้นข้อมูลและการศึกษาพบว่าระบบจำลองขนาดเล็กที่มีอยู่ในปัจจุบันยังไม่สามารถแก้ไข ปัญหาได้อย่างสมบูรณ์โดยเฉพาะในส่วนของการจดจำและตอบสนองต่อป้ายจราจรซึ่งถือเป็นหัวใจสำคัญของการขับเคลื่อนอัตโนมัติ

ความสำคัญของโครงการ

- **เป้าหมาย** : พัฒนารถจำลองขนาดเล็กให้มีประสิทธิภาพในการจดจำและปฏิบัติตามป้ายจราจรให้ดีที่สุดที่จะทำได้
- **ประโยชน์ที่คาดหวัง** : รถจำลองสามารถตอบสนองต่อคำสั่งให้ได้ดีที่สุดที่จะทำได้ เพื่อใช้เป็นเครื่องมือที่สำคัญในการพัฒนาต่อไป

วัตถุประสงค์และขอบเขต

วัตถุประสงค์

- พัฒนาระบบการจดจำป้ายจราจร : เพื่อให้รถจำลองสามารถตรวจจับและแยกแยะป้ายจราจรหลักที่กำหนดไว้ ได้แก่ ป้ายหยุด (Stop), ป้ายเลี้ยวขวา (Right), ป้ายเลี้ยวซ้าย (Left) และป้ายเดินหน้า (Forward)
- พัฒนาระบบควบคุมตัวรถและมอเตอร์ : เพื่อพัฒนาระบบที่สามารถควบคุมการทำงานของตัวรถและมอเตอร์ให้เป็นไปตามคำสั่งที่ได้รับจากการประมวลผลของระบบ
- พัฒนารถให้เคลื่อนไหวตามป้ายจราจร : เพื่อให้รถจำลองสามารถประมวลผลภาพจากกล้อง, ตรวจจับป้ายจราจร, และสั่งการให้รถเคลื่อนที่หรือหยุดตามความหมายของป้ายจราจรที่ตรวจพบ

ขอบเขต

- ความสามารถในการเดินตามป้าย : รถจำลองต้องสามารถเดินตามป้ายจราจรที่กำหนดไว้ได้
- ความสามารถในการ detect ป้าย : กล้องต้องสามารถตรวจจับป้ายได้ทุกป้าย

ส่วนที่ 2



1 อุปกรณ์ที่สำคัญของโครงงาน

- main control board
- motor driver board
- Vision & AI

main control board

Raspberry pi 5

- บอร์ดนี้ถูกเลือกใช้เหนือกว่าบอร์ดทางเลือกอื่น เช่น Open MV และ Arduino เนื่องจากมีพลังประมวลผล Deep Learning สูงที่สามารถรันไลบรารี YOLOv8 ได้อย่างมีประสิทธิภาพที่สุด
- Raspberry Pi 5 เป็นคอมพิวเตอร์ขนาดเล็ก (Single-Board Computer) ที่สามารถรองรับการทำงานแบบ Multitasking ได้พร้อมกันหลายส่วน ทั้งการรับวิดีโอสตรีมจากกล้อง, การรันโมเดล AI, และการสั่งงานมอเตอร์ ซึ่งจำเป็นสำหรับงานที่ต้องมีการตอบสนองแบบเรียลไทม์ จึงเป็นทางเลือกในการทำโครงงานมากกว่าบอร์ดอื่นๆ

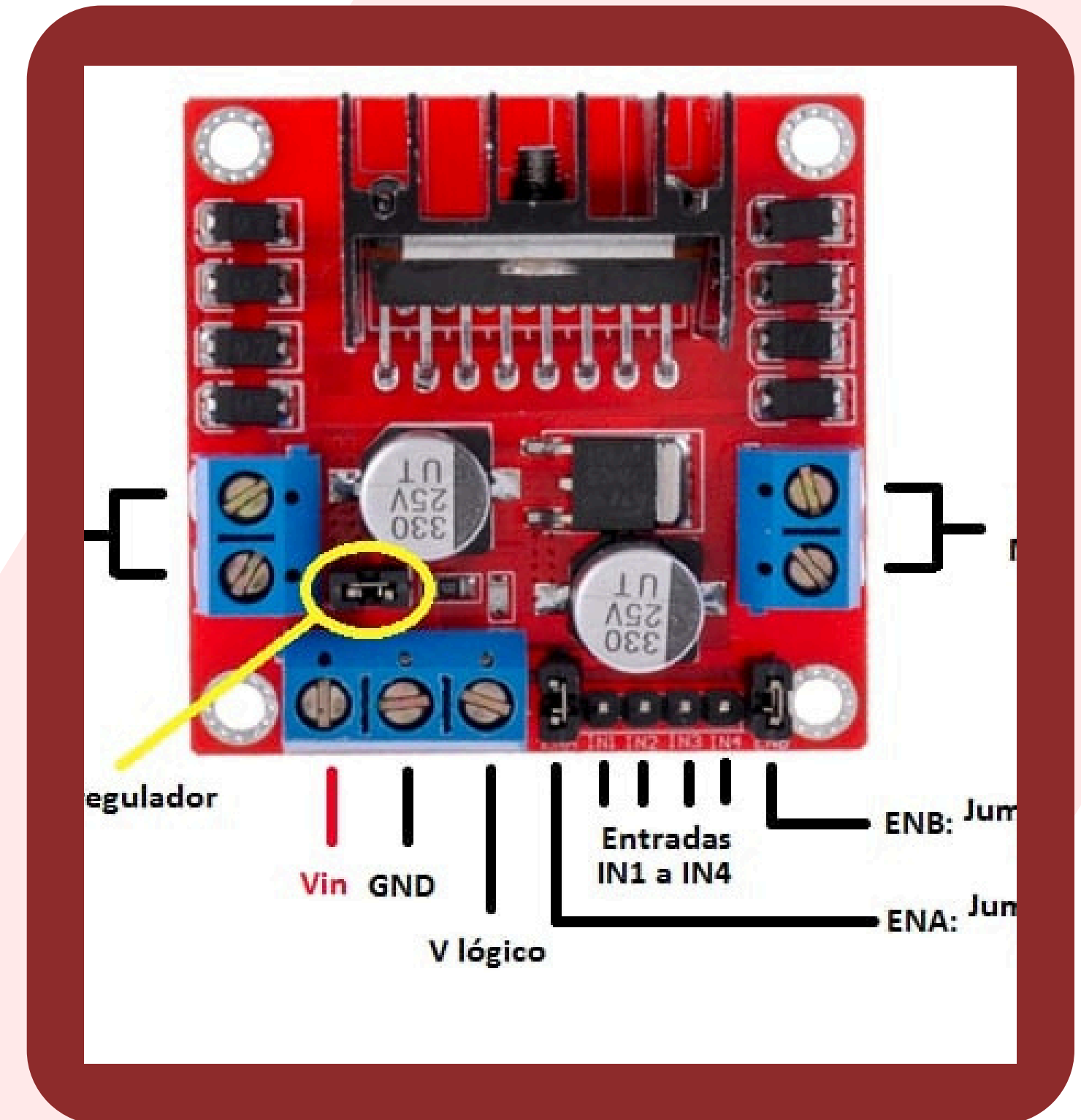
 **Raspberry Pi** **5**



motor driver board

L298N

- ถูกใช้เพื่อทำหน้าที่เป็นสะพาน (Bridge) ในการขยายกระแสไฟฟ้าจากบอร์ด Raspberry Pi ไปยังมอเตอร์กระแสตรง (DC Motor)
- โมดูลนี้รองรับการควบคุมทิศทางและความเร็วของมอเตอร์ 2 ตัว โดยควบคุม ทิศทาง ผ่านขา GPIO และควบคุม ความเร็ว ผ่านสัญญาณ PWM (Pulse Width Modulation)
- มีราคาไม่แพงและหาซื้อง่าย ใช้งานสะดวกกว่าโมดูลขับเคลื่อนตัวอื่นมากกว่าจึงเหมาะในการเอามาทำโครงงานมากที่สุด



การควบคุมความเร็วด้วย PWM และ ควบคุมทิศทาง GPIO

การควบคุมความเร็วด้วย PWM

- Pulse Width Modulation (PWM) คือเทคนิคที่ใช้ในการสร้างสัญญาณดิจิทัลที่มีความกว้างของพัลส์ (Duty Cycle) ที่ปรับเปลี่ยนได้ เพื่อจำลองสัญญาณแอนะล็อก
- ในโครงงานนี้ PWM ถูกใช้เพื่อควบคุมความเร็วรอบของมอเตอร์ โดยการปรับค่า Duty Cycle ที่ขา ENA/ENB ของโมดูล L298N

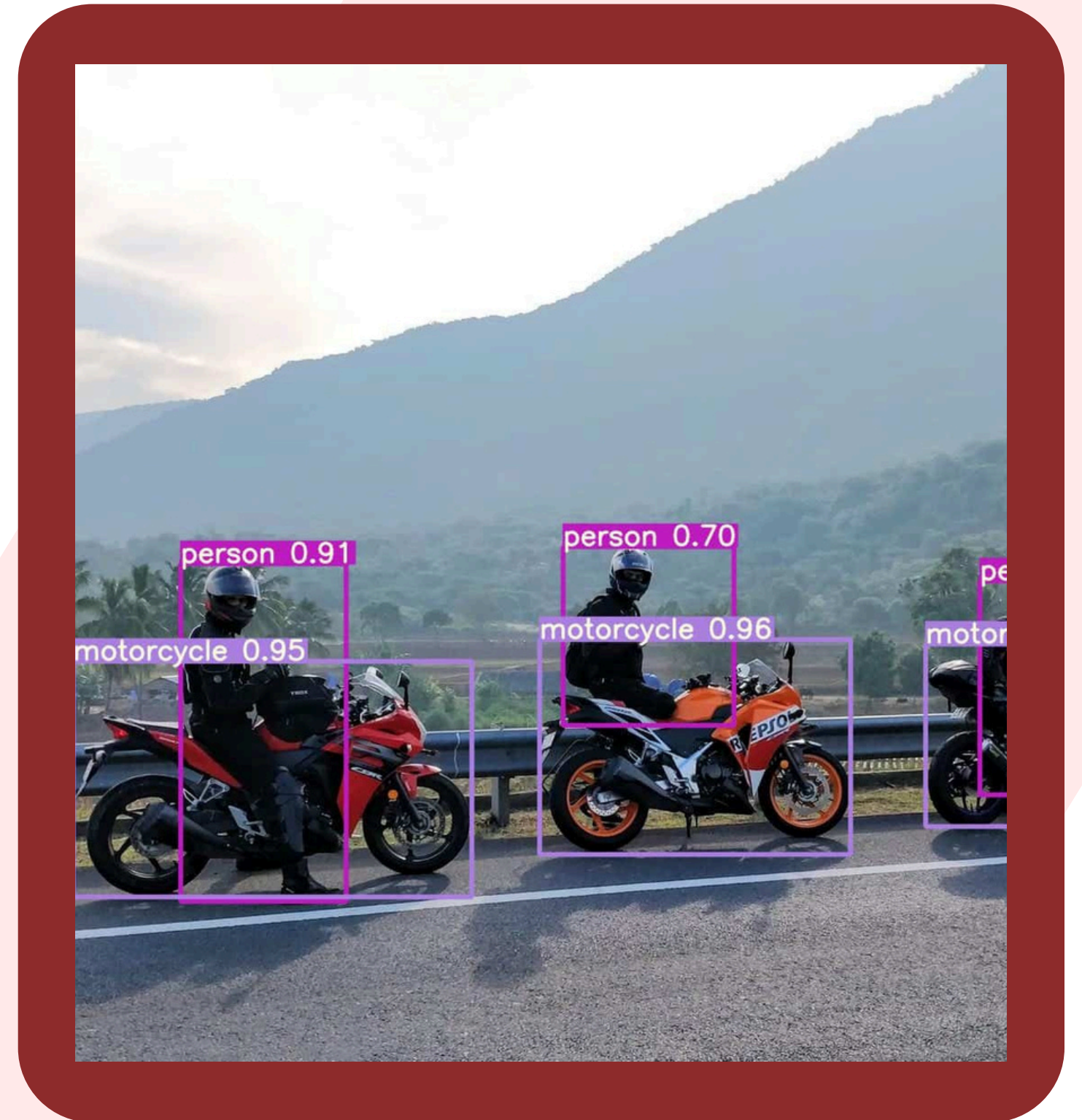
การควบคุมทิศทางด้วย GPIO

- General Purpose Input/Output (GPIO) เป็นขาที่ใช้ในการควบคุมสถานะดิจิทัล (เปิด/ปิด หรือ 0/1)
- ถูกใช้เพื่อกำหนด ทิศทาง (เดินหน้า/ถอยหลัง/เลี้ยว) ของมอเตอร์ โดยการตั้งค่าขาควบคุมอินพุต (In1, In2, In3, In4) ของโมดูล L298N

Vision & AI

YOLOv8

- เป็น Single-Stage Detector ซึ่งหมายความว่าสามารถตรวจจับ (Detect) และจัดประเภท (Classify) วัตถุได้พร้อมกันในการประมวลผลเพียงครั้งเดียว
- เหตุผลในการใช้ YOLOv8 มีความเร็วในการอนุมาน (Inference Speed) ที่สูงมาก เมื่อเทียบกับโมเดลอื่นๆ จึงเหมาะสำหรับโครงการที่ต้องการการตัดสินใจแบบ Real-time ในการขับเคลื่อนรถจำลองตามป้ายจราจร



การเลือกใช้กล้อง

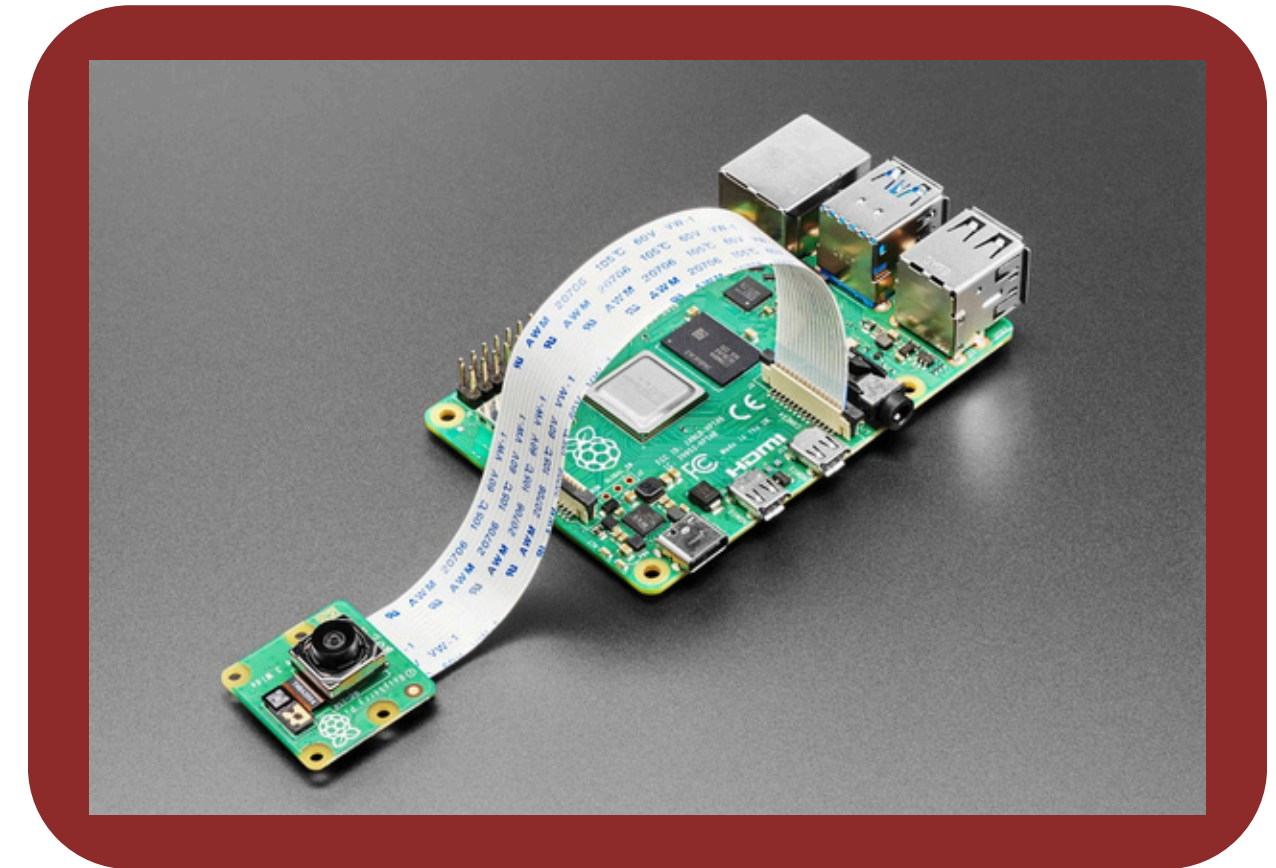
Pi Camera Module

ข้อดีหลัก

- ถูกเลือกใช้เป็นอุปกรณ์หลักเนื่องจากเชื่อมต่อผ่านพอร์ต CSI (Camera Serial Interface) โดยตรงกับ Raspberry Pi

ประสิทธิภาพ

- การเชื่อมต่อแบบ CSI ทำให้มีการรับส่งข้อมูลภาพที่รวดเร็วมากและมีความหน่วงต่ำที่สุดทำให้ลดภาระของ CPU ในการประมวลผลวิดีโอสตรีม และช่วยให้ทรัพยากรส่วนใหญ่ถูกนำไปใช้ในการรันโมเดล AI ได้อย่างเต็มที่



การเลือกใช้กล้อง

Webcam module (USB interface)

ข้อจำกัด

- แม้ Webcam จะมีความยืดหยุ่นสูงในการใช้งาน แต่การเชื่อมต่อผ่านพอร์ต USB มักทำให้เกิดความหน่วง (Latency) ในการรับส่งข้อมูลที่สูงเกินไป

ผลกระทบ

- ความหน่วงที่เพิ่มขึ้นส่งผลให้ความเร็วในการอนุมาน (Inference Speed) ของระบบโดยรวมช้าลง ซึ่งอาจทำให้การตัดสินใจของรถไม่เป็นแบบเรียลไทม์ (Real-time) และเกิดความผิดพลาดในการตอบสนองต่อป้ายจราจรได้สูงมาก

ข้อดีหลัก

- หาซื้อง่าย ต่อใช้งานง่ายไม่ต้องตั้งค่าอะไรมาก



การทำงานร่วมกันของ Vision & AI



1

Input

Pi Camera Module ทำการจับภาพวิดีโอสตรีมด้วยความหน่วงต่ำ แล้วส่งต่อไปยัง Raspberry Pi 5

2

Processing

Raspberry Pi 5 ใช้พลังประมวลผลเพื่อรันโมเดล YOLOv8 ในการวิเคราะห์ภาพที่ได้รับ

การทำงานร่วมกันของ Vision & AI



3

Output

YOLOv8 จะส่งผลลัพธ์ออกมาในรูปของ ID ป้ายจราจร
(เช่น ID 0 = Stop, ID 1 = Left) และ พิกัด ของป้าย

4

Decision Making

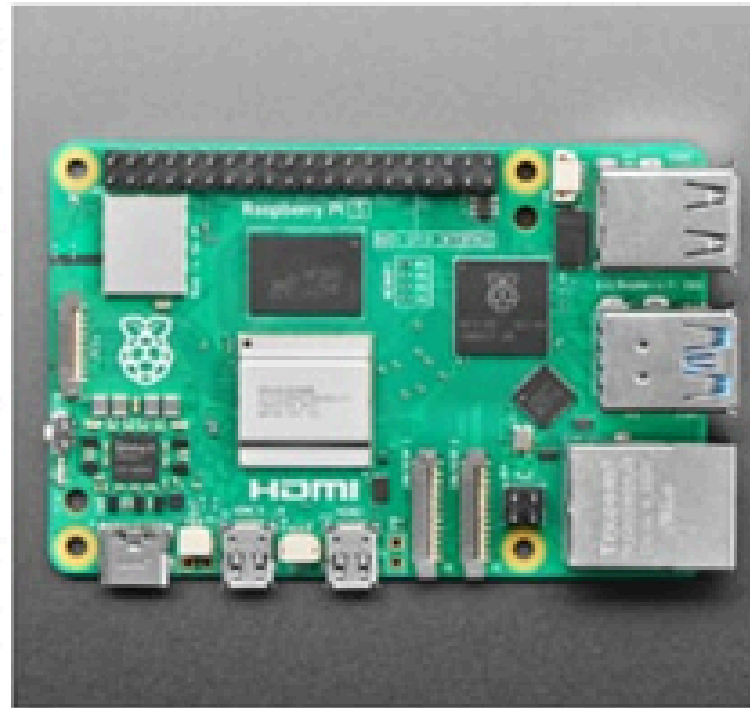
ระบบจะใช้ ID ป้ายที่ตรวจจับได้นี้เป็น คำสั่ง เพื่อส่งต่อ
ไปยังส่วนควบคุมฮาร์ดแวร์ (Motor Control) เพื่อ
สั่งให้รถเคลื่อนที่ตามคำสั่งนั้น ๆ ต่อไป

ส่วนที่ 3

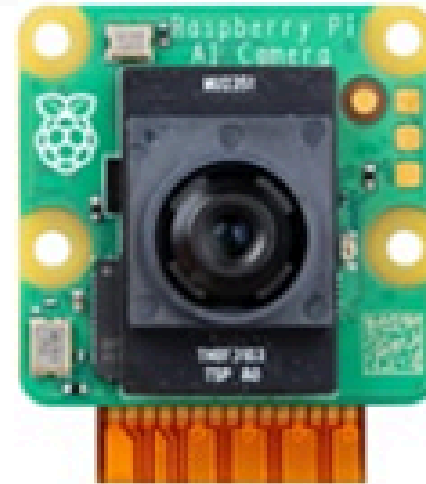


1 Hardware

- บทบาทและการควบคุมของแต่ละอุปกรณ์
- การเชื่อมต่อขา GPIO ระหว่าง Raspberry pi และ L298N
- Movement Logic



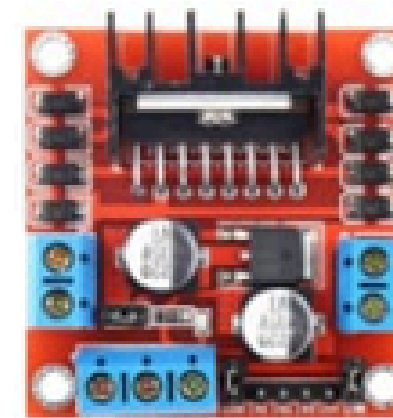
Pi camera Module



camera

Wheel Control

GPIO



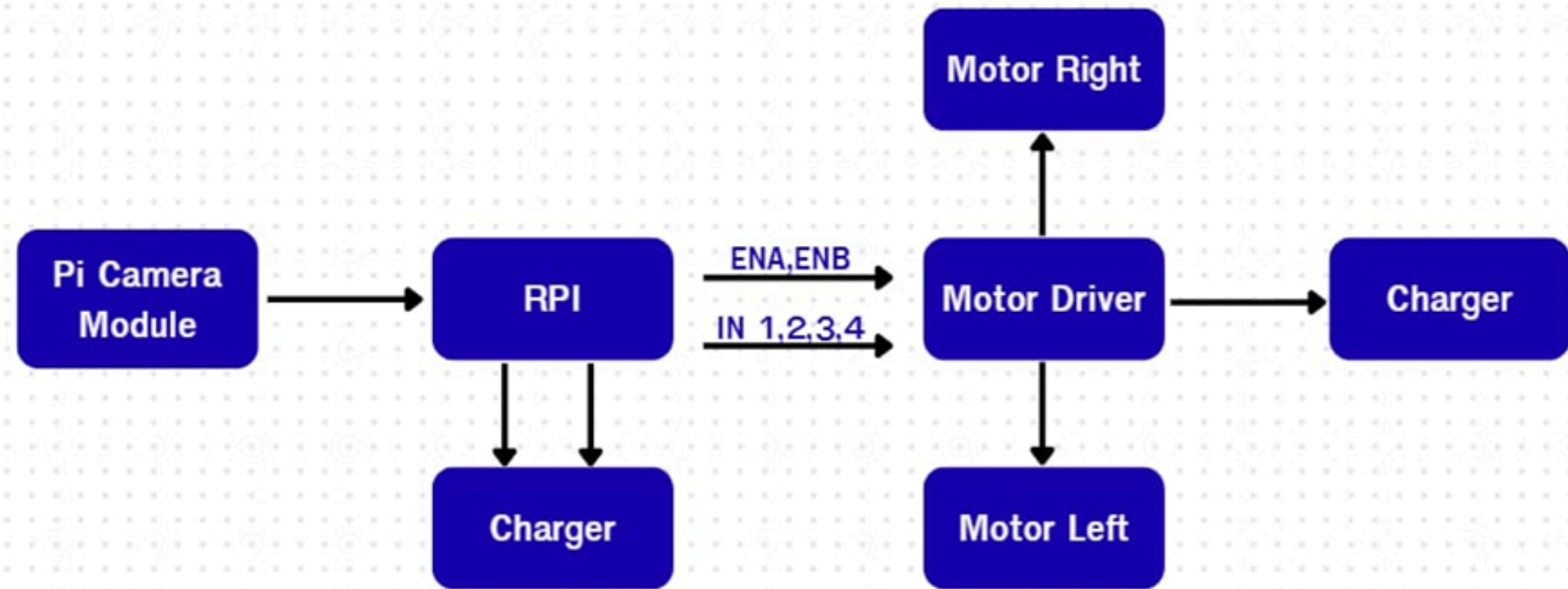
Motor
Driver

DC Motor

DC Motor

บทบาทและการควบคุมของแต่ละอุปกรณ์

องค์ประกอบ	บทบาท	การควบคุม
Raspberry Pi	หน่วยประมวลผล, ควบคุม GPIO/PWM รับ YOLO	ส่วนควบคุมทั้งหมด
Pi Camera Module	เซนเซอร์รับภาพ (Vision Input)	Picamera2
Motor Driver	แปลงสัญญาณลอจิกจาก RPi เป็น กำลังขับเคลื่อนมอเตอร์	ควบคุมขา IN1-IN4 และ ENA/ENB
DC Motors	ล้อขับเคลื่อน	รับกำลังจาก Motor Driver



การเชื่อมต่อขา GPIO ระหว่าง Raspberry pi และ L298N

ขาในโค้ด	หมายเลข GPIO	หน้าที่	ประเภทสัญญาณ
ENA	18	ควบคุมความเร็ว	PWM Output
IN1	17	กำหนดทิศทาง 1	Digital Output
IN2	27	กำหนดทิศทาง 2	Digital Output
ENB	19	ควบคุมความเร็ว	PWM Output
IN3	22	กำหนดทิศทาง 1	Digital Output
IN4	23	กำหนดทิศทาง 2	Digital Output

Movement Logic

การเคลื่อนที่	Motor A (17, 27)	Motor B (22, 23)	การควบคุมความเร็ว (PWM)
เดินหน้า	IN1=LOW, IN2=HIGH (ทิศทาง -1)	IN3=HIGH, IN4=LOW (ทิศทาง +1)	ENA, ENB = SPEED_FWD
ถอยหลัง	IN1=HIGH, IN2=LOW (ทิศทาง +1)	IN3=LOW, IN4=HIGH (ทิศทาง -1)	ENA, ENB = speed
เลี้ยวซ้าย (Pivot)	IN1=HIGH, IN2=LOW (ทิศทาง +1)	IN3=HIGH, IN4=LOW (ทิศทาง +1)	ENA, ENB = SPEED_TURN
เลี้ยวขวา (Pivot)	IN1=LOW, IN2=HIGH (ทิศทาง -1)	IN3=LOW, IN4=HIGH (ทิศทาง -1)	ENA, ENB = SPEED_TURN
หยุด (Brake)	IN1=HIGH, IN2=HIGH	IN3=HIGH, IN4=HIGH	ENA, ENB = 0

ส่วนที่ 4



1 โค้ดโปรแกรม (ใช้ Plcam).

- Motor Control & Setup
- Camera & YOLO Setup
- MainLoop&Navigation Logic
- Code check YOLO

โค้ดของโปรแกรม (ใช้ Plcam). โค้ดควบคุมมอเตอร์และ GPIO (Motor Control & Setup).

กลุ่มโค้ด

การนำเข้าและกำหนดขา/ค่าคงที่

ฟังก์ชัน setup() และ safe_shutdown().

หน้าที่หลัก

นำเข้าไลบรารี RPi.GPIO, time, atexit และกำหนดตัวแปรสำคัญ เช่น หมายเลขขา GPIO (IN1-IN4, ENA, ENB), ความถี่ PWM (PWM_FREQ), และค่าพารามิเตอร์การขับเคลื่อน (SPEED_FWD, TURN_DUR, COMMAND_HOLD)

setup() เตรียมพร้อม GPIO (โหมด BCM, Output) และเริ่มต้นสัญญาณ PWM เพื่อควบคุมความเร็ว
_safe_shutdown() รับประกันว่ามอเตอร์จะหยุดและ GPIO จะถูกทำความสะอาดทุกครั้งเมื่อโปรแกรมสิ้นสุด (ช่วยป้องกันความเสียหาย)

โค้ดควบคุมมอเตอร์และ GPIO (Motor Control & Setup).

กลุ่มโค้ด

ฟังก์ชันขับเคลื่อนรวมๆ

ฟังก์ชันขับเคลื่อนลงละเอียด

หน้าที่หลัก

ได้แก่ `_drive_motor_a()` และ `_drive_motor_b()` ซึ่งเป็นกลไกหลักในการเปลี่ยนทิศทาง (HIGH/LOW บนขา IN) และปรับความเร็ว (Duty Cycle ของ PWM) ของมอเตอร์แต่ละตัว

ได้แก่ `forward()`, `backward()`, `left()`, `right()`, และ `stop()` เป็นคำสั่งสำเร็จรูปที่ใช้ตรรกะของมอเตอร์ A และ B ร่วมกันเพื่อสร้างการเคลื่อนที่ที่ซับซ้อนขึ้น

โค้ดตั้งค่า Vision (Camera & YOLO Setup).

กลุ่มโค้ด

การนำเข้า Vision

parse_args().

การเตรียม YOLO Model

การตั้งค่า Picamera2

หน้าที่หลัก

นำเข้าไลบรารีที่เกี่ยวข้องกับการประมวลผลภาพและ AI ได้แก่ ultralytics.YOLO, picamera2, cv2, และ torch

จัดการการรับพารามิเตอร์จากภายนอก เช่น ชื่อไฟล์ โมเดล , ความละเอียดกล้อง, เฟรมเรต (--fps), และค่าความเชื่อมั่นต่ำสุด (--conf)

โหลดไฟล์โมเดลที่ฝึกมาแล้ว (YOLO(args.model)) และตั้งค่าอุปกรณ์ที่ใช้ประมวลผล (CPU หรือ GPU) มีการรัน Warmup เพื่อเตรียมโมเดลให้ทำงานได้เร็วขึ้นเมื่อเข้าสู่หลัก

ตั้งค่ากล้องตามความละเอียดและเฟรมเรตที่กำหนด (video_config) และเริ่มการทำงานของกล้องเพื่อเตรียมพร้อมรับภาพวิดีโอแบบเรียลไทม์

ไค้ดลูปและการตัดสินใจ (Main Loop & Navigation Logic).

กลุ่มไค้ด

การเริ่มต้นลูปหลัก

การจับภาพและ Inference

ตรรกะการตัดสินใจ (Action Determination).

การสั่งงานมอเตอร์อัจฉริยะ

หน้าที่หลัก

ฟังก์ชัน main() เริ่มต้นด้วยการสั่ง forward(SPEED_FWD) ให้รถเคลื่อนที่ทันที จากนั้นเข้าสู่ลูป while True เพื่อทำงานอย่างต่อเนื่อง

ในแต่ละรอบลูป จะมีการ จับภาพ ล่าสุดจากกล้อง และทำการ YOLO Detection (model()) ตามรอบที่กำหนด (args.skip) เพื่อ ค้นหาป้ายหรือวัตถุ

วิเคราะห์ผลลัพธ์จาก YOLO (Class ID) เพื่อกำหนด action_name ("stop", "left", "right", "go") กลุ่มไค้ดนี้มีตรรกะพิเศษ สำหรับการ ตรวจสอบ Class ID 2 (right) ซ้ำด้วย Conf. ต่ำ เพื่อดูว่าควรเปลี่ยน ไปสั่ง Class ID 1 (left) แทนหรือไม่

ตรวจสอบว่าคำสั่งที่ตัดสินใจได้ถูกส่งซ้ำที่เกินไปหรือไม่ (โดยใช้ COMMAND_HOLD) หากผ่านเงื่อนไข จะ เรียกใช้ฟังก์ชันมอเตอร์ที่เกี่ยวข้อง:

ส่วนที่ 5



1 โค้ดโปรแกรม (ใช้ Webcam)

- Setup & Imports
- Motor Control
- YOLO Inference
- Main Loop

โค้ดของโปรแกรม (ใช้ Webcam).

โค้ดส่วนการตั้งค่าและการนำเข้า (Setup & Imports).

กลุ่มโค้ด

ส่วนการนำเข้าไลบรารี

การกำหนดพิน GPIO

การปรับพฤติกรรมรถ

การตั้งค่า Argument/AI

หน้าที่หลัก

เตรียมเครื่องมือที่ใช้ควบคุมพิน, จัดการเวลา, ประมวลผลภาพ, และใช้โมเดล AI

กำหนดหมายเลขพินสำหรับควบคุมทิศทาง (IN) และความเร็ว (ENA/ENB) ของมอเตอร์

ตั้งค่าความเร็วเริ่มต้น, ระยะเวลาเลี้ยวซ้าย, และเวลาหน่วงคำสั่งซ้ำเพื่อป้องกันมอเตอร์ทำงานหนัก

กำหนดพารามิเตอร์ที่รับจากบรรทัดคำสั่ง (เช่น ชื่อโมเดล, ความละเอียดกล้อง) และแมป Class ID ของ AI ไปเป็นชื่อคำสั่ง ("stop", "go")

โค้ดส่วนควบคุมมอเตอร์ (Motor Control).

กลุ่มโค้ด

ฟังก์ชันตั้งค่าเริ่มต้น

ฟังก์ชันควบคุมมอเตอร์ระดับต่ำ

ฟังก์ชันการเคลื่อนที่

ฟังก์ชันหยุดและปิดระบบ

หน้าที่หลัก

ตั้งค่าโหมด GPIO, กำหนดพินเป็นเอาต์พุต, เริ่มต้น PWM และลงทะเบียนฟังก์ชันปิดระบบ (_safe_shutdown)

ควบคุมทิศทาง (HIGH/LOW บน IN1-IN4) และความเร็ว (Duty Cycle ของ PWM) ของมอเตอร์ A และ B แยกกัน

คำสั่งการเคลื่อนที่ที่เรียกใช้ฟังก์ชันควบคุมมอเตอร์ระดับต่ำ เพื่อให้รถ เดินหน้า, ถอยหลัง, หรือ เลี้ยวแบบ Pivot (หมุนอยู่กับที่)

สั่งหยุดมอเตอร์ (Brake/ปล่อยไหล) และรับประกันว่าพิน GPIO ทั้งหมดจะถูกปล่อย (GPIO.cleanup()) เมื่อโปรแกรมสิ้นสุด

โค้ดส่วนการประมวลผล AI (YOLO Inference).

กลุ่มโค้ด

การโหลด/Warmup โมเดล

การรับภาพจากกล้อง

การทำ Inference

ตรรกะการเลือกป้าย

หน้าที่หลัก

โหลดไฟล์โมเดล AI และเตรียมพร้อมสำหรับการประมวลผลจริง

เปิดกล้องและอ่านเฟรมภาพทีละเฟรมในลูปหลัก

ส่งภาพให้โมเดล YOLO ทำการตรวจจับป้ายจราจร

ดึงค่าความเชื่อมั่นทั้งหมดที่ตรวจพบ แล้วเลือก ป้ายที่มีความ
เชื่อมั่นสูงสุด เพื่อกำหนดเป็น action_name

โค้ดส่วนลูปการทำงานหลัก (Main Loop).

กลุ่มโค้ด

ลูปไม่สิ้นสุด

การจัดการ Skip Frame

การป้องกันคำสั่งซ้ำ

การสั่งมอเตอร์ตามป้าย

การแสดงผล

หน้าที่หลัก

ทำซ้ำขั้นตอนทั้งหมดของการรับภาพ,
ประมวลผล, และสั่งการมอเตอร์

ตรวจสอบว่าควรข้ามการประมวลผล AI ในเฟรมปัจจุบันหรือไม่
(เพื่อลดภาระ CPU)

ตรวจสอบว่าคำสั่งล่าสุดซ้ำกับคำสั่งปัจจุบันหรือไม่ ถ้าซ้ำและยังไม่พ้น
เวลาหน่วง (COMMAND_HOLD) จะคงสถานะคำสั่งเดิมไว้

สั่งการเคลื่อนที่จริงตาม action_name ที่ได้จาก AI (เช่น
ถ้าเป็น "left" ให้เลี้ยวซ้ายๆ แล้วกลับมาเดินหน้าต่อ)

แสดงภาพจากกล้อง พร้อม Bounding Box
และคำสั่งปัจจุบัน (ถ้าเปิดโหมด --show)

ส่วนที่ 6



- 1 การทำ Autorun
 - เตรียมไฟล์ Python
 - ตั้งค่า Autorun ด้วย `systemd`
- 2 เปรียบเทียบ code

การทำ Autorun 1 เตรียมไฟล์

ขั้นตอน

เพิ่ม Shebang

การเพิ่มบรรทัดพิเศษที่อยู่บนสุดของไฟล์ เพื่อบอกระบบปฏิบัติการว่าควรใช้โปรแกรม แปลภาษาตัวใดในการรันไฟล์นั้น

ตั้งค่าสิทธิ์ให้ไฟล์รันได้

เป็นการบอกระบบว่าไฟล์นี้เป็นไฟล์ที่สามารถเรียกใช้งานได้

วิธีทำ

เข้า Terminal ใช้คำสั่ง `nano` เพื่อเปิดไฟล์ใส่คำสั่ง `#!/usr/bin/env python3` เพิ่มบนบรรทัดแรกสุดของ โค้ดรถเดินเดินตามป้ายหรือโค้ดที่จะทำ Autorun แล้วกดออก

ใช้คำสั่งเปลี่ยนโหมด `chmod` เพื่อเพิ่มสิทธิ์และ `+x` เป็นการเพิ่มไฟล์ที่จะทำ Autorun ตามด้วย พารามิเตอร์ของไฟล์ที่จะทำ ทั้งหมดทำใน Terminal

การทำ Autorun 2 ตั้งค่า Autorun ด้วย systemd

ระบบจัดการดูแลให้โปรแกรมเริ่มต้นทำงานอย่างถูกต้องในลำดับเวลาที่เหมาะสม และดูแลให้ทำงานต่อไปเรื่อย ๆ หากเกิดข้อผิดพลาด

ขั้นตอน

สร้าง Service File

ใส่เนื้อหาลงใน Service File

เปิดใช้งานและเริ่ม Service

ตรวจสอบสถานะ และ รีบูตเพื่อทดสอบ
Autorun

วิธีทำ

สร้างไฟล์ sudo nano ตั้งค่า ชื่อ yolo-motor.service
ขึ้นมา ไฟล์นี้จะอยู่ในไดเรกทอรี /etc/systemd/system/
ซึ่งเป็นที่เก็บไฟล์ Service ของระบบ systemd

เพิ่มชุดคำสั่งและคำอธิบายในรูปแบบข้อความ ที่ใช้สำหรับ
กำหนดวิธีการทำงานของโปรแกรมให้แก่ระบบจัดการ Service

ใส่ 3 คำสั่ง 1 โหลด Service ใหม่ 2 เปิดใช้งานให้รับอัตโนมัติเมื่อบูต
3 เริ่ม Service ทันที เพื่อทดสอบ

ตรวจสอบว่าทำงานถูกต้องหรือไม่ด้วยคำสั่ง sudo systemctl
status yolo-motor.service ถ้าเห็นสถานะ Active: active
(running) แสดงว่าสำเร็จ ทำการรีบูตด้วยคำสั่ง sudo reboot

เปรียบเทียบ code

Webcam camera

ใช้ได้กับกล้องทั่วไป หาซื้อ่ายติดตั้งง่าย เพราะ cv2 เป็นไลบรารีมาตรฐาน แต่มี ความหน่วงสูง มีการดึง CPU มาใช้ในการสตรีมรับภาพ จึงทำให้รัน YOLO ได้ไม่เต็มที่

ไม่มีตรรกะอะไรเลย เจอป้ายไหนเลี้ยวทันที จึงมีข้อดีคือทำงานไวแต่ข้อเสียคือผิดพลาดเยอะมาก

Pi camera

เป็นไลบรารี official ของ Raspberry Pi สำหรับกล้องรุ่นใหม่ ประสิทธิภาพสูงกว่าและมีความหน่วง (Latency) ต่ำกว่า Webcam เยอะมาก ติดตั้งยากและต้องใช้ไลบรารีเฉพาะของ Picamera2

มีตรรกะพิเศษในการแยกแยะป้าย โดยเฉพาะป้ายที่คล้ายกันเช่น ป้ายเลี้ยวขวาและเลี้ยวซ้าย ทำให้ทำงานช้าลง แต่แม่นยำมากกว่า

ส่วนที่ 7



1

ผลการทดสอบ

- ผลการทดสอบสั่งงานมอเตอร์โดยตรง
- ผลการทดสอบการตอบสนองของรถต่อป้ายจราจร
- ผลการทดสอบสถานการณ์การเคลื่อนที่โดยรวม

Unit test

ผลการทดสอบผลงานมอเตอร์โดยตรง

est	ฟังก์ชันที่ ทดสอบ	ทดสอบครั้งที่ 1- 10 สำเร็จ	ทดสอบครั้งที่ 11- 20 สำเร็จ	ทดสอบครั้งที่ 21 30สำเร็จ
1	สั่งเดินหน้า	8	9	9
2	สั่งเลี้ยวซ้าย	9	7	8
3	สั่งเลี้ยวขวา	6	10	8
4	สั่งหยุด	8	8	10

Unit test

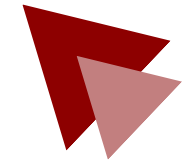
ผลการทดสอบการตอบสนองของรถต่อป้ายจราจร

Test	ฟังก์ชันที่ทดสอบ	ทดสอบครั้งที่ 1-10 สำเร็จ	ทดสอบครั้งที่ 11-20 สำเร็จ	ทดสอบครั้งที่ 21-30 สำเร็จ
1	เจอบ้าย เดินหน้า	9	8	10
2	เจอบ้ายเลี้ยว ซ้าย	4	7	6
3	เจอบ้ายเลี้ยว ขวา	6	10	8
4	เจอบ้ายหยุด	10	10	10

Systems test

ผลการทดสอบสถานการณ์การเคลื่อนที่โดยรวม

Test	ฟังก์ชันที่ ทดสอบ	ทดสอบครั้งที่ 1- 10 สำเร็จ	ทดสอบครั้งที่ 11- 20 สำเร็จ	ทดสอบครั้งที่ 21- 30 สำเร็จ
1	ไปข้างหน้าแบบ ไม่มีป้าย	10	10	10
2	ไปข้างหน้ามี ป้ายตรงไป	9	7	7
3	เจอป้ายเลี้ยว ขวา	4	7	6
4	เจอป้ายเลี้ยว ซ้าย	7	8	5
5	เจอป้ายหยุด	10	8	10



THANK YOU